

1. Tömbök használata a szerveroldali adatszerkezetekben

A szerveroldali programozásban (pl. egy Node.js API-ban) az adatokat gyakran memóriában tároljuk tömbök formájában, mielőtt tényleges adatbázist (mint a MySQL vagy MongoDB) kötnénk be. Ezek a tömbök objektumokat tartalmaznak (JSON-szerű struktúra), ahol minden objektum egy-egy rekordnak (pl. egy felhasználónak vagy terméknek) felel meg. Fontos megérteni, hogy a kliens (frontend) kéréseire a szerver ezekből a tömbökből fog adatokat kikeresni, módosítani vagy újakat hozzáadni.

- **Példa 1:** Memória-alapú adatbázis (Mock DB) szimulálása egy tömbbel.

```
// Mock (szimulált) adatbázis a szerver memóriájában
const users = [
  { id: 1, username: "admin", role: "admin", isActive: true },
  { id: 2, username: "kovacsp", role: "user", isActive: true },
  { id: 3, username: "kiss_zita", role: "user", isActive: false }
];

// Egy egyszerű funkció, ami szimulálja az összes adat lekérését
function getAllUsers() {
  console.log("Adatbázis lekérdezése folyamatban...");
  return users;
}
```

- **Példa 2:** Új adat hozzáadása a szerveroldali adatszerkezethez.

```
// Új felhasználó regisztrációjának szimulálása
function registerUser(newUsername) {
  const newUser = {
    // Dinamikus ID generálás (az eddigi hossz alapján)
    id: users.length + 1,
    username: newUsername,
    role: "user",
    isActive: true
  };

  users.push(newUser); // Hozzáadjuk a tömbhöz (memóriába mentés)
  console.log(`Sikeres regisztráció: ${newUser.username}`);
  return newUser;
}
```

2. Adatok transzformálása .map() segítségével

Amikor a szerver adatokat küld a kliensnek (pl. a React frontendnek), ritkán küldjük el a nyers adatbázis-rekordot. Miért? Mert lehetnek benne jelszavak, belső azonosítók, vagy egyszerűen más formátumra van szükség. A .map() metódus tökéletes arra, hogy a nyers adatokból egy új, 'megtisztított' vagy formázott tömböt hozzunk létre, ami már biztonságosan elküldhető a hálózaton.

- **Példa 1:** Szenzitív adatok (pl. jelszavak) eltávolítása válaszadás előtt (Data Transfer Object - DTO minta).

```
const dbUsers = [
  { id: 1, name: "Péter", passwordHash: "xyz123", email: "p@p.hu" },
  { id: 2, name: "Anna", passwordHash: "abc987", email: "a@a.hu" }
];

// Csak azokat az adatokat tartjuk meg, amit a frontend láthat (DTO - Data Transfer Object)
const safeUsersForFrontend = dbUsers.map(user => {
  return {
    id: user.id,
    name: user.name,
    email: user.email
  }; // A passwordHash-t szándékosan kihagyjuk!
});

console.log(safeUsersForFrontend);
```

- **Példa 2:** Adatok kiegészítése kalkulált mezőkkel (pl. adó hozzáadása egy webshop backendjén).

```
const products = [
  { id: 1, name: "Laptop", netPrice: 200000 },
  { id: 2, name: "Egér", netPrice: 5000 }
];

const TAX_RATE = 0.27; // 27% ÁFA

const productsWithGrossPrice = products.map(product => {
  // Bővítjük a meglévő objektumot egy új mezővel
  return {
    ...product, // Spread operátor: átmásoljuk a meglévő adatokat (id, name, netPrice)
    grossPrice: Math.round(product.netPrice * (1 + TAX_RATE))
  };
});

console.log(productsWithGrossPrice);
```

3. Adatszűrés szerveroldalon .filter() segítségével

Egy API végpontnak gyakran kell szűrt adatokat visszaadnia. Például a frontend csak az 'aktív' felhasználókat kéri, vagy a webshopban a vásárló csak a '10 000 Ft alatti' termékekre keres rá. A szerveroldalon a .filter() segítségével tudjuk a hatalmas adattömböt a feltételeknek megfelelően leszűkíteni. A .filter() mindig új tömböt ad vissza.

- **Példa 1:** Státusz szerinti szűrés (pl. csak a raktáron lévő termékek).

```
const inventory = [
  { id: 101, name: "Monitor", inStock: true },
  { id: 102, name: "Billentyűzet", inStock: false },
  { id: 103, name: "Webkamera", inStock: true }
];

// Szimulált API végpont logika: GET /api/products/available
const availableProducts = inventory.filter(item => item.inStock === true);
console.log("Elérhető termékek:", availableProducts);
```

- **Példa 2:** Összetett keresés (Keresőmező és árkategória alapján).

```
const cars = [
  { brand: "Toyota", price: 5000000, type: "Sedan" },
  { brand: "Suzuki", price: 3000000, type: "Hatchback" },
  { brand: "Ford", price: 7000000, type: "SUV" }
];

// Tegyük fel, hogy a kliens ezeket a paramétereket küldte (Query paraméterek)
const maxPrice = 6000000;
const searchType = "Sedan";

// Többszörös feltétel: csak azok kelljenek, amik beleférnek a keretbe VAGY a típus egyezik
const searchResults = cars.filter(car => {
  return car.price <= maxPrice || car.type === searchType;
});
```

4. Egyetlen elem megkeresése ID alapján tömbben

A REST API-k alapvető feladata, hogy ha a kliens kér egy adott erőforrást (pl. egy felhasználó profilját a 'GET /users/5' végponton), akkor a szerver megtalálja azt az egyetlen elemet az azonosítója (ID) alapján. Erre a `.find()` való. Fontos: a `.find()` az első találatot (egy objektumot) adja vissza, míg a `.filter()` egy tömböt. Ha nincs találat, a `.find()` 'undefined' értéket ad, amit a szervertől feltétlenül kezelniünk kell (pl. 404 Not Found hiba küldésével).

- **Példa 1:** Adatbázis rekord keresése ID alapján.

```
const employees = [
  { id: 1, name: "Gábor", department: "IT" },
  { id: 2, name: "Zita", department: "HR" },
  { id: 3, name: "Bence", department: "IT" }
];

const targetId = 2; // Ezt az ID-t kaptuk a kienstől az URL-ben
```

```
// A find megáll, amint megtalálta az első olyat, ahol a feltétel true
const employee = employees.find(emp => emp.id === targetId);

console.log(employee.name); // "Zita" (objektumot kaptunk vissza)
```

- **Példa 2:** Hibakezelés (404 Not Found szimuláció), ha az elem nem létezik.

```
const requestedId = 99; // Nincs ilyen ID

const foundEmployee = employees.find(emp => emp.id === requestedId);

// Kulcsfontosságú a serveroldalon: vizsgálni kell, hogy lett-e eredmény!
if (!foundEmployee) {
  // Ha undefined, akkor a kliens hibás adatot kért
  console.error(`Hiba: 404 - A ${requestedId} ID-jű dolgozó nem található!`);
  // Express.js-ben ez így nézne ki: res.status(404).json({ error: "Not found" });
} else {
  console.log("Megtalálva:", foundEmployee);
}
```

5. Összegzés és statisztika készítés `.reduce()` segítségével

Sokszor a server feladata, hogy dashboardokhoz (vezérlőpultokhoz) vagy riportokhoz adatokat aggregáljon. Nem a kliensre bízuk több ezer adat összeadását, hanem a server kiszámolja, és csak a végeredményt küldi át. A `.reduce()` metódussal összeadhatunk árakat, megszámlolhatunk kategóriákat, vagy bonyolult statisztikákat (pl. átlag, leggyakoribb elem) építhetünk fel egyetlen lépésben.

- **Példa 1:** Pénzügyi összegzés (pl. egy rendelés végösszegének kiszámítása a backend-en).

```
const orderItems = [
  { product: "Laptop", price: 350000, quantity: 1 },
  { product: "Egér", price: 10000, quantity: 2 },
  { product: "Táska", price: 20000, quantity: 1 }
];

// acc (akkumulátor): a részeredmény, item: az aktuális elem. A 0 a kezdőérték.
const totalOrderValue = orderItems.reduce((acc, item) => {
  return acc + (item.price * item.quantity);
}, 0);

console.log(`A rendelés végösszege: ${totalOrderValue} Ft`); // 390000 Ft
```

- **Példa 2:** Haladó statisztika: Csoportosítás (Hány termék van egy-egy kategóriában?).

```
const devices = [
  { name: "iPhone", type: "Phone" },
```

```

    { name: "Samsung", type: "Phone" },
    { name: "MacBook", type: "Laptop" },
    { name: "Dell", type: "Laptop" },
    { name: "Asus", type: "Laptop" }
  ];

  // Az akkumulátor most egy üres objektum {} lesz, amibe a statisztikát építjük!
  const stats = devices.reduce((acc, device) => {
    // Ha még nincs ilyen típus a statisztikánkban, létrehozuk 0 értékkel
    if (!acc[device.type]) {
      acc[device.type] = 0;
    }
    // Megnöveljük a darabszámot
    acc[device.type]++;

    return acc; // Visszaadjuk a részeredményt a következő iterációnak
  }, {}); // <- A {} a kezdőérték

  console.log(stats); // { Phone: 2, Laptop: 3 }

```

6. Immutabilitás és tömbműveletek gyakorlása

A modern JavaScriptben (főleg React és komolyabb Node.js alkalmazások esetén) arany szabály az immutabilitás (változtathatatlanág). Ez azt jelenti, hogy sosem módosítjuk (mutáljuk) az eredeti tömböt vagy objektumot (kerüljük a push, pop, splice metódusokat), hanem mindig egy újat (másolatot) hozunk létre a módosított adatokkal. Miért? Mert a rejtett adatmódosulások (side-effects) okozzák a legnehezebben megtalálható hibákat (bugokat). Erre a legjobb eszköz a Spread operátor (...).

- **Példa 1:** Elem biztonságos hozzáadása a tömbhöz (push HELYETT spread operátor).

```

const baseUsers = ["Anna", "Béla"];

// ROSSZ MEGOLDÁS (Mutáció): baseUsers.push("Cecília");

// JÓ MEGOLDÁS (Immutabilitás):
// Létrehozunk egy teljesen új tömböt, amibe "kibontjuk" (...) a régit, és
// hozzátesszük az újat.
const updatedUsers = [...baseUsers, "Cecília"];

console.log(baseUsers); // ["Anna", "Béla"] -> Az eredeti érintetlen maradt!
console.log(updatedUsers); // ["Anna", "Béla", "Cecília"]

```

- **Példa 2:** Meglévő elem adatának módosítása a tömbben immutábilisan (.map segítségével).

```

const todos = [
  { id: 1, task: "Tanulni", completed: false },

```

```

    { id: 2, task: "Aludni", completed: false }
  ];

  // A feladat: A 2-es ID-jú teendőt 'completed: true'-ra állítani!
  const updatedTodos = todos.map(todo => {
    if (todo.id === 2) {
      // Ha ez az az elem, amit keresünk, akkor készítünk belőle egy módosított
      MÁSOLATOT
      return { ...todo, completed: true };
    }
    // Ha nem ez az, akkor simán visszaadjuk az eredetit
    return todo;
  });

  console.log(updatedTodos[1].completed); // true (a másolatban)
  console.log(todos[1].completed); // false (az eredetiben)

```

7. Node.js CommonJS (require) vs ES Modules (import)

Ahogy a backend kódunk nő, több fájlra kell bontanunk (moduláris programozás). A Node.js története során két rendszer alakult ki. A régebbi a CommonJS, amely a 'require()' függvényt és a 'module.exports' objektumot használja (ez a Node.js hagyományos módszere). A modernebb az ES Modules (ESM), amely a frontendről már ismert 'import' és 'export' kulcsszavakat használja. Ma már az ES Modules az ipari standard, de Node.js-ben a package.json fájlban be kell állítani a 'type': 'module' sort a használatához.

- **Példa 1:** A klasszikus CommonJS módszer (Régi Node.js szabvány).

```

// --- math.js (Exportáló fájl) ---
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;

// Objektumba csomagolva exportáljuk
module.exports = {
  add,
  subtract
};

// --- app.js (Importáló fájl) ---
// Beolvassuk a modult a require() segítségével
const math = require('./math.js');

console.log(math.add(5, 3)); // 8

```

- **Példa 2:** A modern ES Modules (ESM) módszer (Új ipari szabvány).

```

// --- FIGYELEM: A package.json-ben kell lennie egy "type": "module" sornak! ---

```

```
// --- utils.js (Exportáló fájl) ---
// A 'export' kulcsszót egyenesen a függvény elé írjuk (Named export)
export const multiply = (a, b) => a * b;

export const greet = (name) => `Hello ${name}!`;

// --- server.js (Importáló fájl) ---
// Destrukturálással (kapcsos zárójelekkel) importáljuk a konkrét függvényeket
import { multiply, greet } from './utils.js';

console.log(greet("Diákok")); // "Hello Diákok!"
console.log(multiply(4, 5)); // 20
```

8. Node.js beépített modulok (fs, path)

A Node.js óriási előnye a sima böngészős JavaScripthez képest, hogy hozzáfér az operációs rendszerhez! Ehhez beépített modulokat biztosít. Két nagyon fontos modul a 'path' (útvonalak kezelésére) és az 'fs' (File System - fájlrendszer kezelésére). A 'path' modul azért kritikus, mert a Windows visszaperjelet (\), míg a Linux/Mac perjelet (/) használ a mappák között. A path.join() megoldja, hogy a kódunk minden operációs rendszeren hiba nélkül fusson.

- **Példa 1:** A path modul használata biztonságos mappa-útvonalak létrehozásához.

```
import path from 'path';

// A folyamat (process) aktuális munkakönyvtára
const currentDir = process.cwd();

// Biztonságos összeillesztés (NE használjunk string összefűzést fájlutakhoz!)
// WINDOWS-on ez lesz: C:\MyProject\data\users.json
// LINUX-on ez lesz: /home/user/MyProject/data/users.json
const dataFilePath = path.join(currentDir, 'data', 'users.json');

console.log("Fájl pontos útvonala:", dataFilePath);
```

- **Példa 2:** Az fs (File System) modul szinkron használata (könyvtár tartalmának olvasása).

```
import fs from 'fs';
import path from 'path';

const logsDir = path.join(process.cwd(), 'logs');

try {
  // fs.readdirSync: SZINKRON függvény.
  // FIGYELEM: A szinkron fájlkezelés megállítja (blokkolja) a teljes szerveret amíg le
```

```
nem fut!
// Csak a szerver elindulásakor (pl. konfiguráció beolvasása) szabad használni!
const files = fs.readdirSync(logsDir);
console.log("A logs mappában lévő fájlok:", files);

} catch (error) {
  console.error("Nem sikerült beolvasni a mappát:", error.message);
}
```

9. Aszinkron fájlkezelés (fs.promises)

Mivel a Node.js egyszálú (egy fő szálon szolgálja ki az összes felhasználót), tilos a fájlrendszerrel végzett lassú műveletekkel (olvasás/írás) blokkolni a szerveret. Képzeljük el, ha egy nagy fájl mentése 2 másodpercig tart, addig egyetlen más felhasználó kérését sem tudná feldolgozni az oldal! Erre a megoldás az aszinkron fájlkezelés, kifejezetten az 'fs/promises' almodul, ami Promise-alapúvá teszi a fájlkezelést, így használhatjuk vele a modern, tiszta async/await szintaxist.

- **Példa 1:** JSON adatfájl beolvasása aszinkron módon (nem blokkolja a szerveret).

```
import { promises as fs } from 'fs'; // A promises almodult importáljuk
import path from 'path';

async function loadUsers() {
  const filePath = path.join(process.cwd(), 'users.json');

  try {
    console.log("Fájl olvasásának indítása...");
    // Az await miatt a kód itt megáll (de a szerver főszála TUD MÁST CSINÁLNI közben!)
    const fileContent = await fs.readFile(filePath, 'utf-8');

    // A fájlból beolvasott szöveget (String) JavaScript tömbbé/objektummá alakítjuk
    const usersArray = JSON.parse(fileContent);
    console.log(`Sikeresen beolvasva ${usersArray.length} felhasználó.`);

    return usersArray;
  } catch (error) {
    console.error("Hiba a fájl olvasásakor:", error.message);
  }
}
```

- **Példa 2:** Adatok aszinkron kiírása fájlba (pl. naplózás vagy adatok mentése).

```
import { promises as fs } from 'fs';
import path from 'path';
```

```
async function saveLogMessage(message) {
  const logFilePath = path.join(process.cwd(), 'server.log');
  const timestamp = new Date().toISOString();
  const logEntry = `[${timestamp}] ${message}\n`;

  try {
    // fs.appendFile: Nem felülírja a fájlt, hanem a végéhez fűzi a szöveget
    // Az await itt is biztosítja a nem-blokkoló végrehajtást
    await fs.appendFile(logFilePath, logEntry, 'utf-8');
    console.log("Log bejegyzés sikeresen mentve a háttérben.");
  } catch (error) {
    console.error("Kritikus hiba: Nem sikerült a fájlba írni!", error);
  }
}

saveLogMessage("A szerver sikeresen elindult a 3000-es porton.");
```